

LMLS Lab.

(Large-scale Machine Learning Systems Lab.)

Machine Learning

Large-Scale Machine Learning

Sunwoo Lee

sunwool@inha.ac.kr



INHA UNIVERSITY



Today, we will talk about...

What is large-scale ML and why do we care of it?



Do not share or distribute these slides!

저작권 안내 / Copyright Notice

본 사이트에서 수업 자료로 이용되는 저작물은 저작권법 제25조 수업목적저작물 이용 보상금제도의 의거, 한국복제전송저작권협회와 약정을 체결하고 적법하게 이용하고 있습니다. **약정범위를 초과하는 사용은 저작권법에 저촉될 수 있으므로 수업자료의 대중 공개·공유 및 수업 목적외의 사용을 금지합니다. (일부 공개로 제공되는 유튜브 강의를 불법으로 공유하거나 배포하는 행위를 금함)**

The teaching materials provided on this site are copyrighted and signed legally by the Korea Reproduction and Transmission Rights Association. Using these materials beyond the scope of the agreement is a violation against copyright laws. **Hence, one is not allowed to open or share the materials in public. Using them outside of the class is strictly prohibited.**

2020. 3. 11.

인하대학교·한국복제전송저작권협회

Inha University · Korea Reproduction and Transmission Rights Association

- ***Why Large-Scale?***
- Scaling: How and What?
- Scaling ML Solutions
- Real-World Large-Scale ML

BigData Era

- Data are getting generated in everywhere around us!



Internet of Things (IoT)



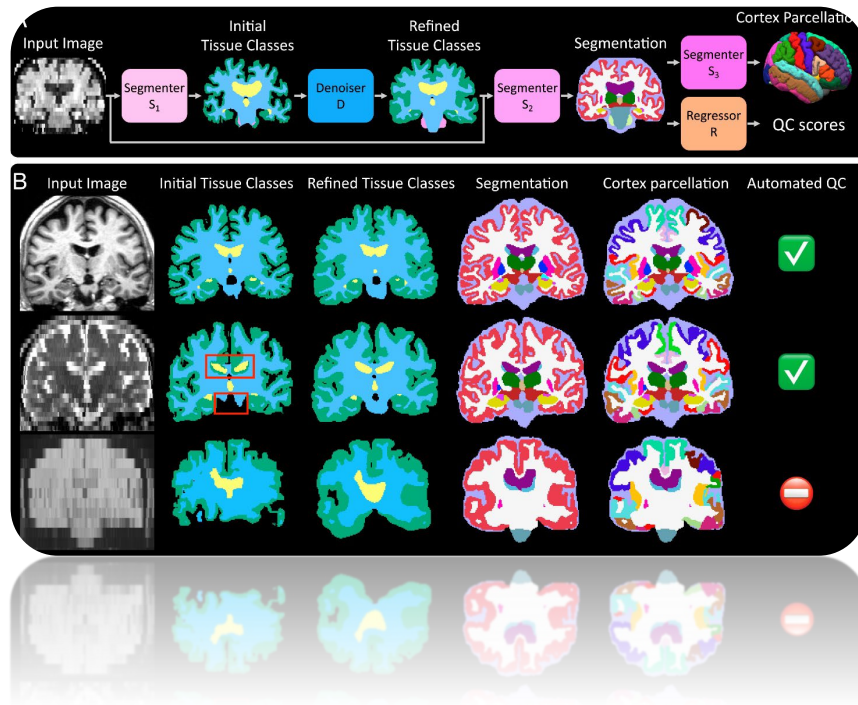
Social Network Service (SNS)



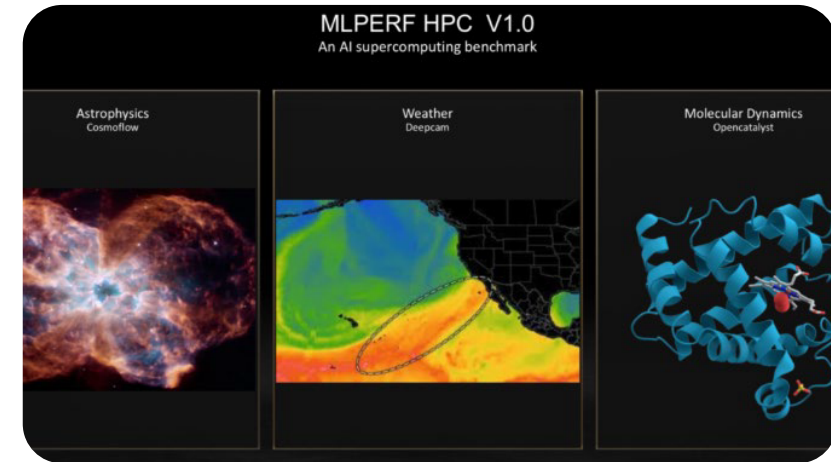
Powerful H/W

Large-Scale ML Provides Unprecedented Opportunities!

Medical Data Analysis



Scientific Applications



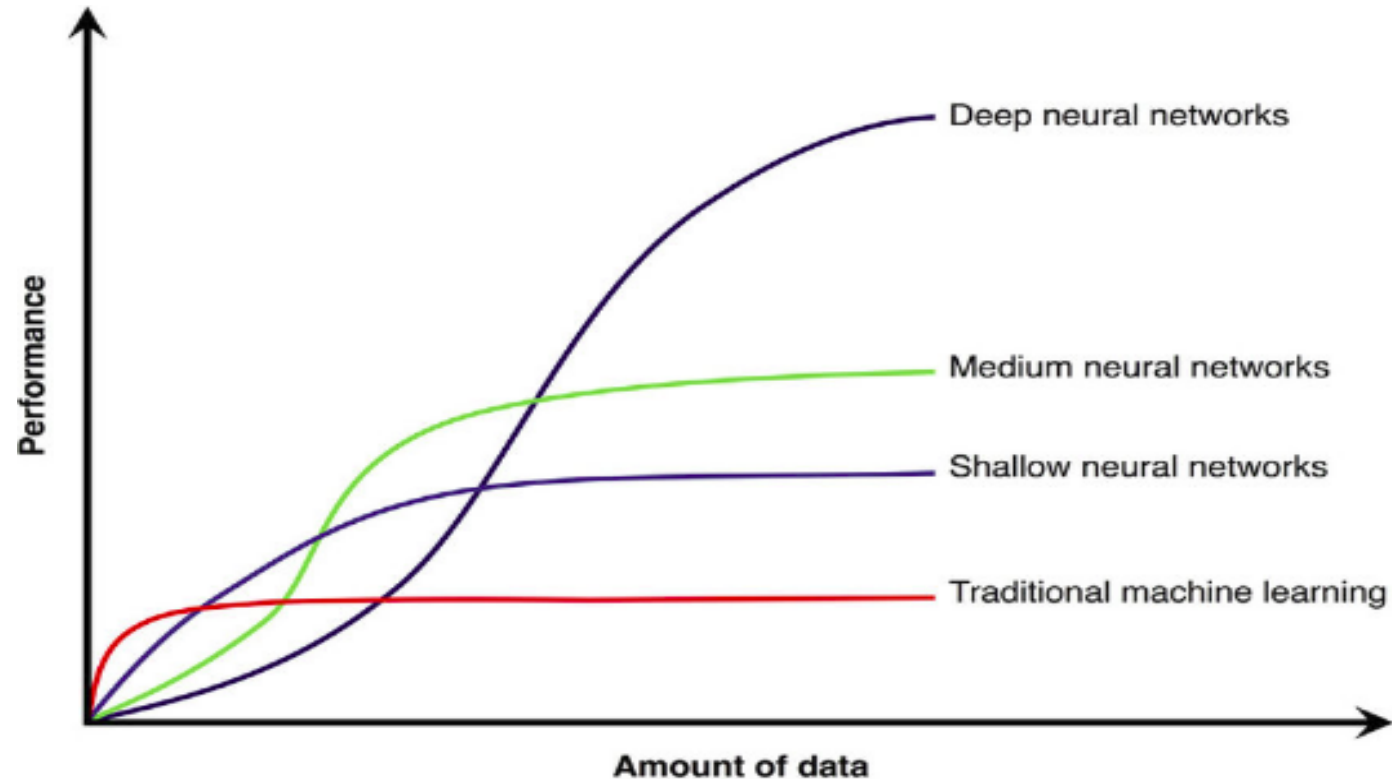
User-Level Services



Why Should We Go for Large-Scale?

- Large-scale data provides *new insights*.
 - E.g., large-scale SNS data provides societal trends.
- ***Data-intensive applications*** for human.
 - E.g., recommendation systems.
- A large-scale datasets enable ML to achieve ***better generalization***.
 - E.g., The overfitting can be mitigated by having more data.

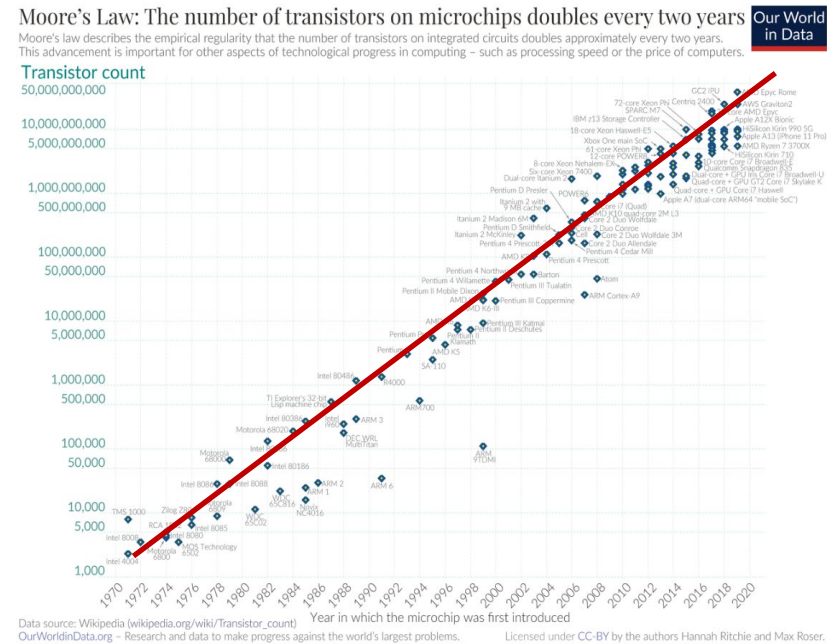
The Larger The Model, The Better The Performance



- Why Large-Scale?
- ***Scaling: How and What?***
- Scaling ML Solutions
- Real-World Large-Scale ML

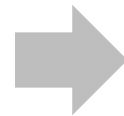
Moore's Law Broken

- Chip scaling has nearly stopped (Moore's law has been broken).



- That is, to further improve the performance, we need to use **multiple** systems.

Modern Multi-Core Systems



Because the capability of each core is not improved much, CPUs tend to provide more cores instead!

Parallelization

- Given multiple computing resources, then, how do we make use of them? **Parallelize** algorithms!

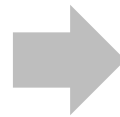
Sequential Program

```
1 function findMin(a,b)
2   var c = 10, d = 0
3   NO:
4   var e = 10, f = 0
5   function findMin(a,b)
6     function findMin(a,b)
7     function findMin(a,b)
8     function findMin(a,b)
9     function findMin(a,b)
10    function findMin(a,b)
```



Parallel Program

```
1 function findMin(a,b)
2   var c = 10, d = 0
3   NO:
4   var e = 10, f = 0
5   function findMin(a,b)
6     function findMin(a,b)
7     function findMin(a,b)
8     function findMin(a,b)
9     function findMin(a,b)
10    function findMin(a,b)
```



Re-design algorithm
(parallelization)

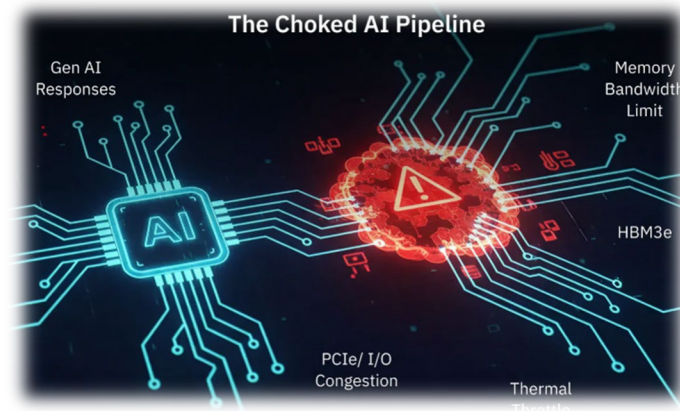


Okay, Then, What Matters? (1/2)

- **Scalability**: How efficiently can an algorithm be parallelized?
- Amdahl's law
 - $\text{Speedup} = \frac{1}{(1-P) + \frac{P}{S}}$
 - P : the time proportion of the algorithm that can be parallelized
 - S : the speedup factor
- Example
 - If 50% of the whole workload can be parallelized and the speedup of the parallel part is 4×, the speedup is $\frac{1}{(1-0.5) + \frac{0.5}{4}} = 1.6$.

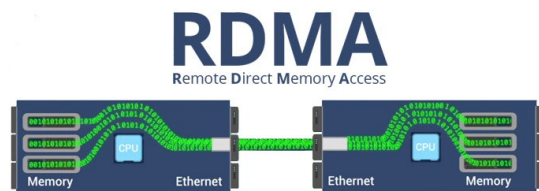
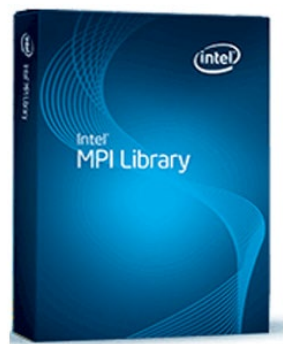
Okay, Then, What Matters? (2/2)

- Why isn't the full workload parallelized?
 - Algorithm
 - Communication cost
- Usually, algorithms are not designed considering the scalability.
 - They rather focus on whether it solves the problem or not.
 - Thus, we should focus on the parallelizable part of the algorithm.
- **Communication cost** is the most practical bottleneck in large-scale applications.



Parallelization Strategies

Custom Parallelization
(User-Level Software, E.g., MPI, RDMA, etc)



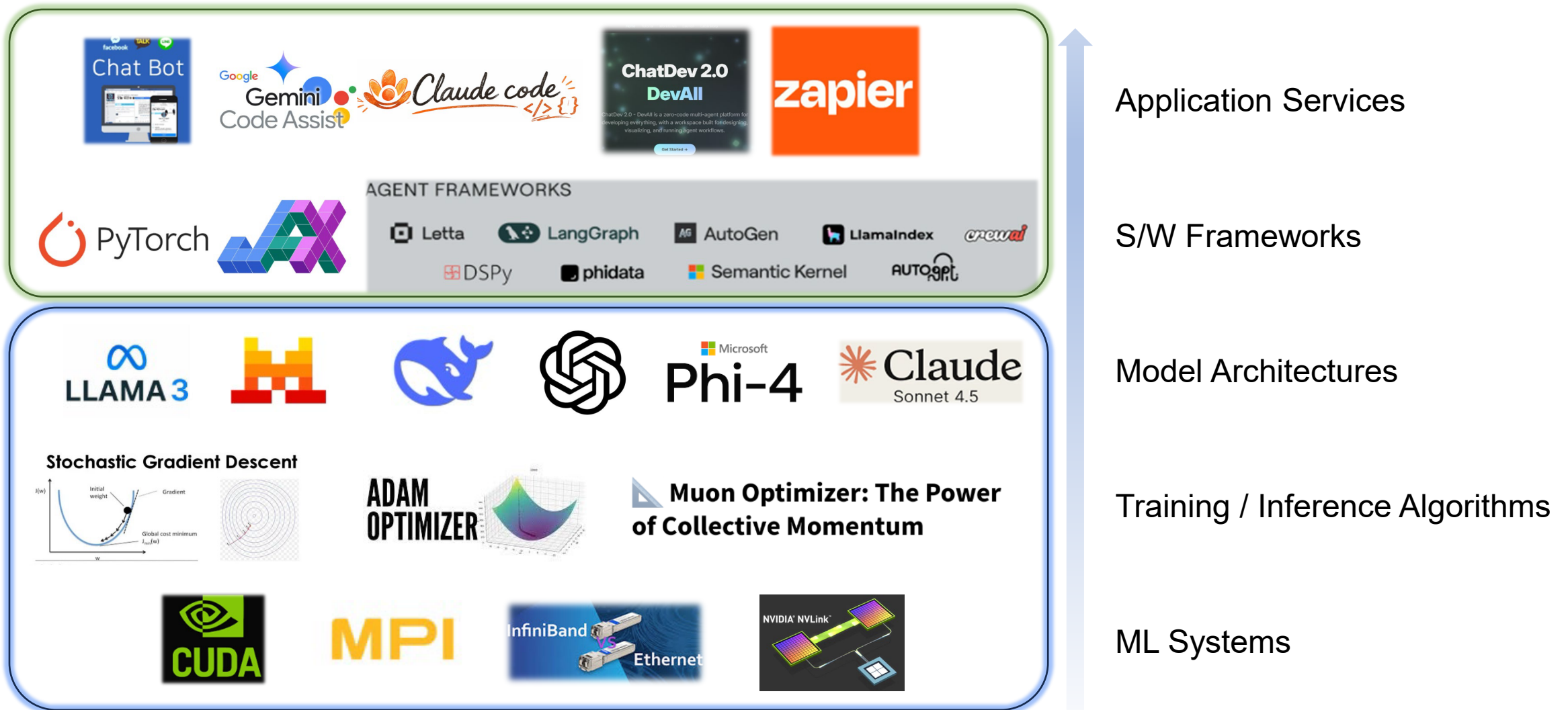
Cloud Computing



BigData Frameworks



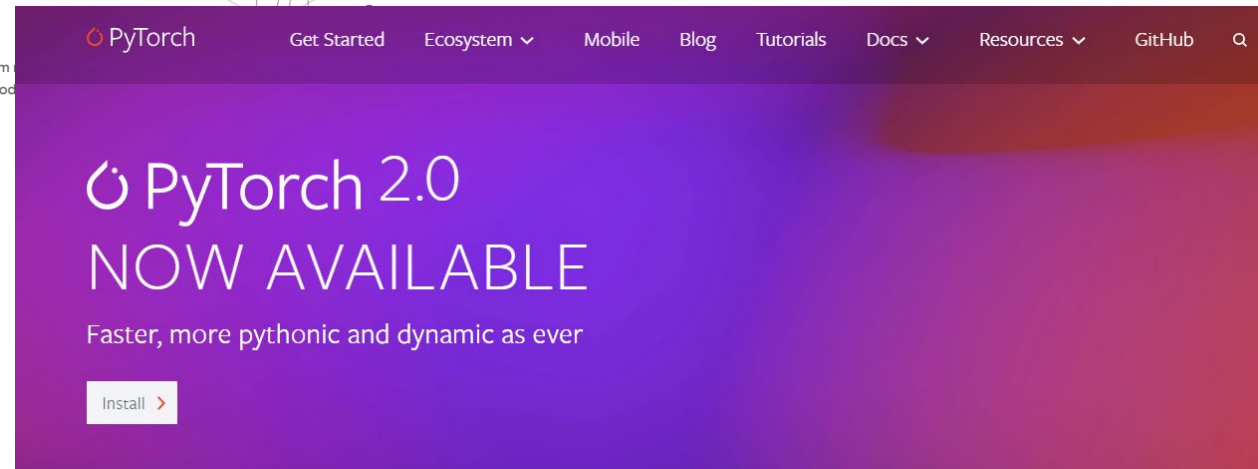
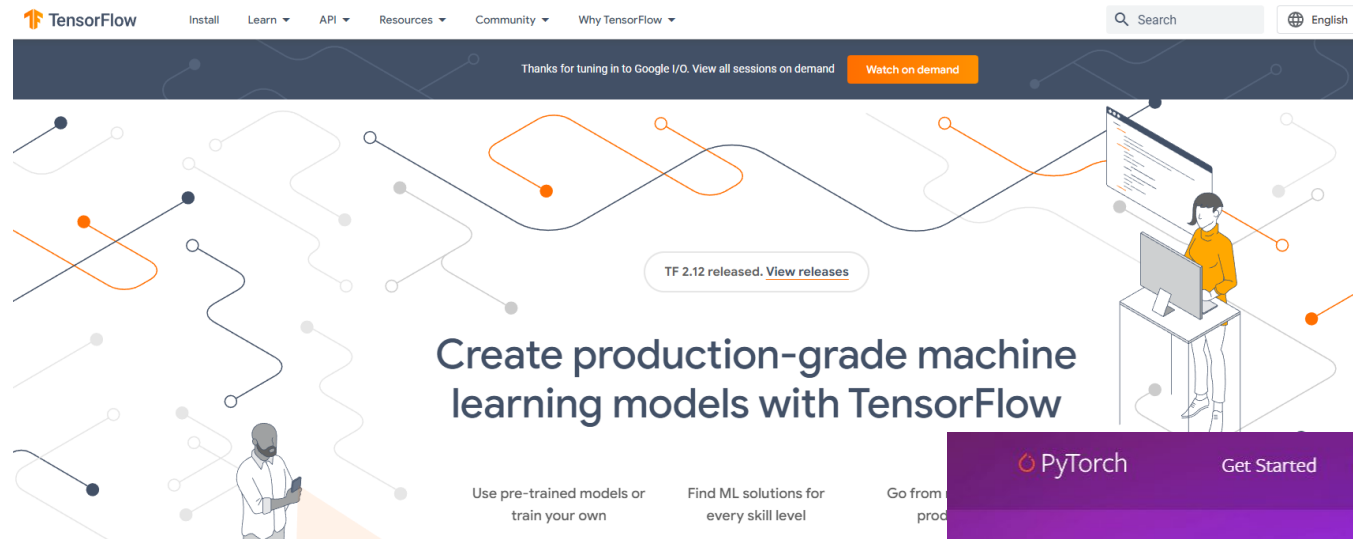
Modern View of Large-Scale AI S/W



- Why Large-Scale?
- Scaling: How and What?
- ***Scaling ML Solutions***
- Real-World Large-Scale ML

Software Frameworks for ML

Open-Source Software Frameworks.



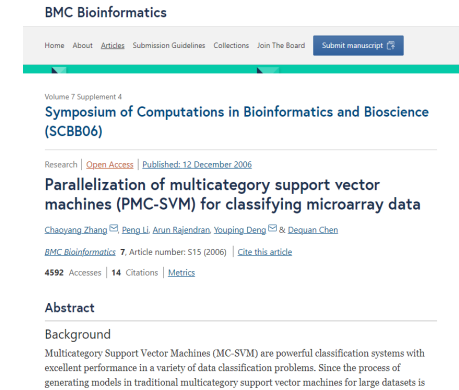
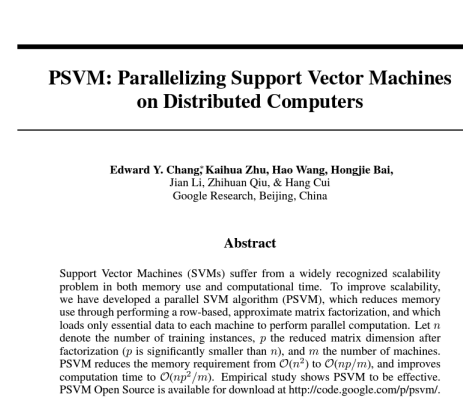
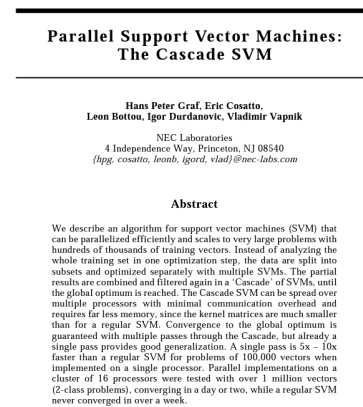
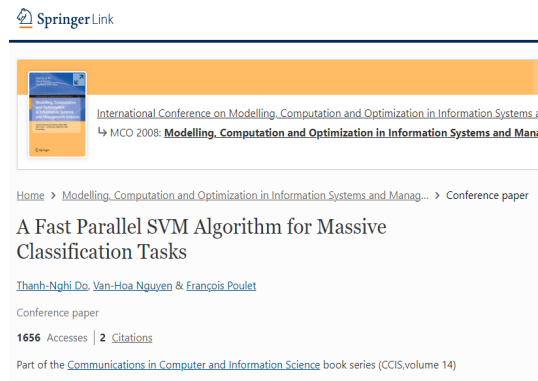
Software Parallelization

- Algorithm's perspective
 - Algorithm design / choice
 - Algorithm complexity (scalability)
 - Data consumption pattern
- System's perspective
 - System design / choice
 - Implementation details
 - Communication patterns



Scaling ML Algorithms (1/3)

- What's my algorithm's complexity? (Is it scalable?)
 - The portion of workload with a constant complexity.



E.g., when parallelizing SVM, the bottleneck is QP programming. To scale up SVM, usually, a parallelized version of approximation method is employed, such as .

Scaling ML Algorithms (2/3)

- Some algorithms causes practical issues in scale.
 - E.g., Large-scale K-Means can cause an extremely expensive I/O cost issue.

Parallel K-Means Clustering Based on MapReduce

Weizhong Zhao^{1,2}, Huifang Ma^{1,2}, and Qing He¹

¹ The Key Laboratory of Intelligent Information Processing, Institute of Computing Technology, Chinese Academy of Sciences
² Graduate University of Chinese Academy of Sciences
[zhaozw,mahf,heq]@ict.ac.cn

Abstract. Data clustering has been received considerable attention in many applications, such as data mining, document retrieval, image segmentation and pattern classification. The enlarging volumes of information emerging by the progress of technology, makes clustering of very large scale of data a challenging task. In order to deal with the problem, many researchers try to design efficient parallel clustering algorithms. In this paper, we propose a parallel k -means clustering algorithm based on MapReduce, which is a simple yet powerful parallel programming technique. The experimental results demonstrate that the proposed algorithm can scale well and efficiently process large datasets on commodity hardware.

Keywords: Data mining; Parallel clustering; K -means; Hadoop; MapReduce.

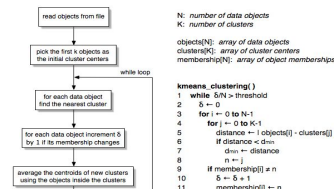
Parallel K-Means Data Clustering

This software package [parallel-kmeans.tar.gz](https://github.com/zhaozw/parallel-kmeans) (4.6 MB) of parallel K-means data clustering contains the followings:

- A parallel implementation using [OpenMP](#) and C
- A parallel implementation using [MPI](#) and C
- A sequential version in C

For large data support (more than 2 billion number of data points), see [this page](#) for an MPI implementation that uses 8-byte integers.

Algorithm:



Computers & Industrial Engineering
Volume 152, February 2021, 107023



Parallel batch k-means for Big data clustering

Rasim M. Alguliyev, Ramiz M. Alguliyev, Lyudmila V. Sukhostat, [R](#), [S](#)

Show more

+ Add to Mendeley [Share](#) [Cite](#)

<https://doi.org/10.1016/j.cie.2020.107023>

[Get rights and content](#)

Abstract

The application of clustering algorithms is expanding due to the rapid growth of data volumes. Nevertheless, existing algorithms are not always effective because of high computational complexity. A new parallel batch clustering algorithm based on the k -means algorithm is proposed. The proposed algorithm splits a dataset into equal partitions and reduces the exponential growth of computations. The goal is to preserve

Massively Parallel k -Means Clustering for Perturbation Resilient Instances

Vincent Cohen-Addad^{1,2}, Vahab Mirzadeh^{1,2}, Pratin Zhang^{1,2}

Abstract

We consider k -means clustering of n data points in Euclidean space in the Massively Parallel Computation (MPC) model, a computational model which is an abstraction of modern massively parallel computing system such as MapReduce. Recent work provides evidence that getting $O(1)$ -approximate k -means solution for general input points using $c(\log n)$ rounds in the MPC model may be impossible under certain conditions (Chaffin, Kato & Ullrich 2019). However, the real-world data points usually have better structures. One instance of interest is the set of data points which is perturbation resilient (Bilu & Linial 2019). In particular, a point set is ϵ -perturbation resilient for k -means if perturbing pairwise distances by multiplicative factors in the range $[1, 1 + \epsilon]$ does not change the optimum k -means clusters. We bypass the worst case lower bound by considering the perturbation resilient input points and showing $c(\log n)$ rounds k -means clustering algorithms for these instances in the MPC model. Specifically, we show a fully scalable $(1 + \epsilon)$ -approximate k -means clustering algorithm for $O(n)$ -perturbation resilient

missing massive datasets. As a canonical clustering problem, k -means clustering is defined as follows: Given a dataset where each data element is represented by a vector of real-valued features, the goal is to find k representative vectors, called centers, such that the sum of squared distances from each input vector to the closest center is minimized. Similarly, in k -median clustering, the goal is to minimize sum of distances to closest centers. k -means and k -median have become essential building blocks for unveiling hidden patterns and extracting information in datasets, especially in the unsupervised clustering contexts where supervised machine learning cannot be applied, or little is known about the data or when the dataset is massive and the competitive supervised methods become impractical.

These clustering problems have received a lot of attention through the years and for almost half a century. Hence, good progress on our understanding of the complexity of the problems has been made. We know that the problems are hard to approximate within a factor better than 1.1 in high-dimensional Euclidean space and admits approximation schemes in low-dimension (Aronis et al., 1996; Kallipouridis & Rao, 2007; Cohen-Addad, 2018; Cohen-Addad et al., 2019a). We further show of constant factor approximation algorithms for high-dimensional Euclidean space, better than for general metric space (Cohen-Addad et al., 2019b).

Scaling ML algorithms is challenging!

Scaling ML Algorithms (3/3)

- Deep Learning solutions are usually parallelized twice.
 - Shared-Memory Programming Model
 - Distributed-Memory Programming Model



Shared-Memory Programming Model:
Multiple cores share the memory space.

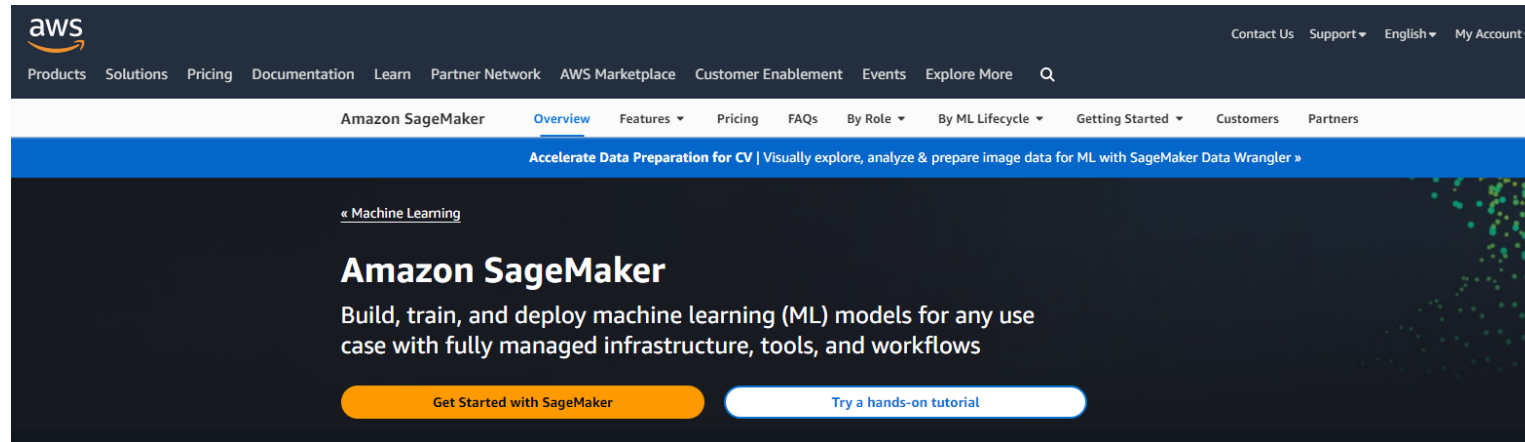
+



Distributed-Memory Programming Model:
Multiple processes perform inter-process
communications to share local results.

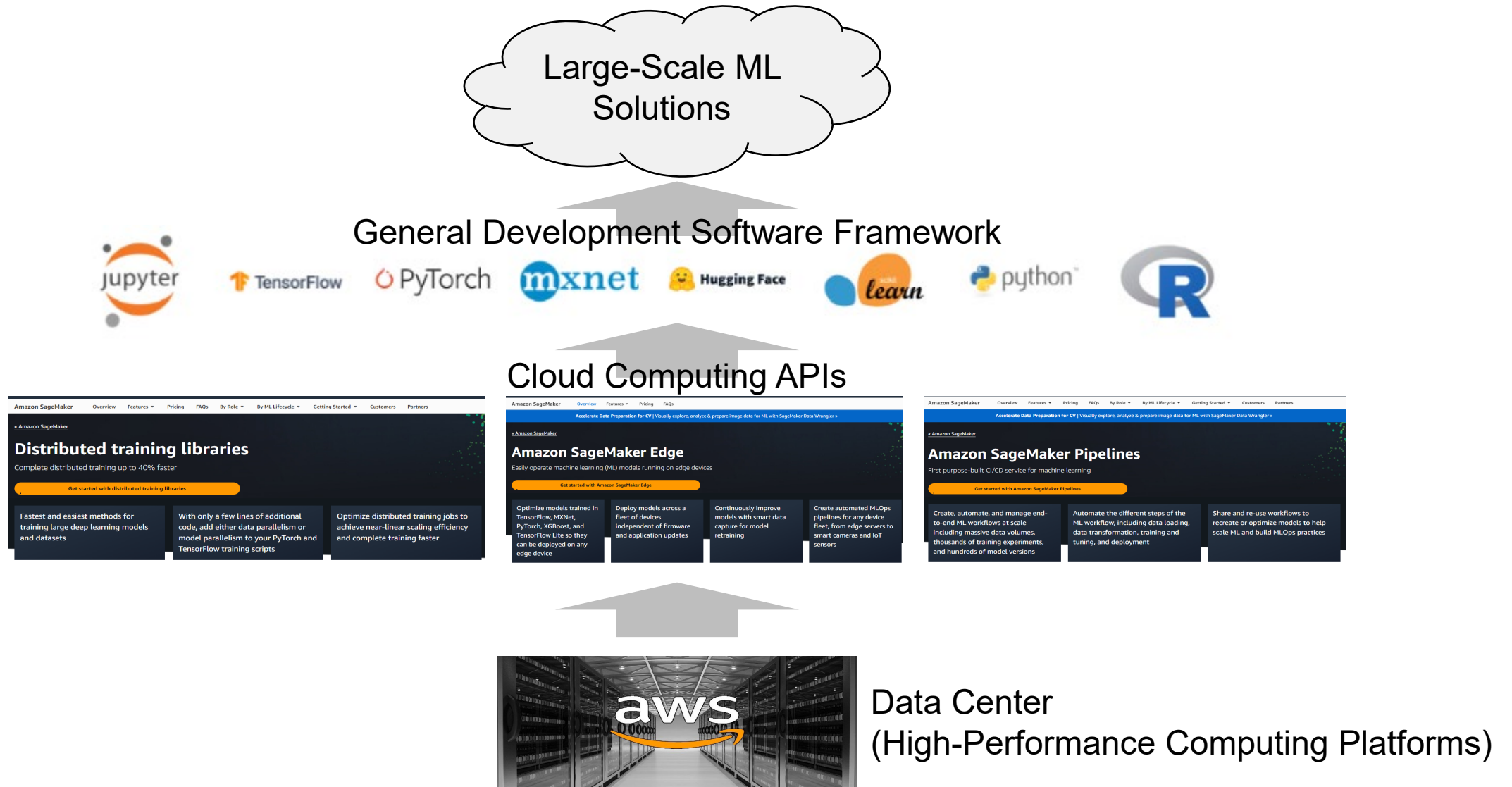
ML on Cloud Computing Environments

- Cloud Computing Services provide a set of useful APIs to build up ML solutions and deploy them quickly.



- MLOps: a general methodology that is built on applying DevOps practices to ML workloads.

Example: AWS ML Services

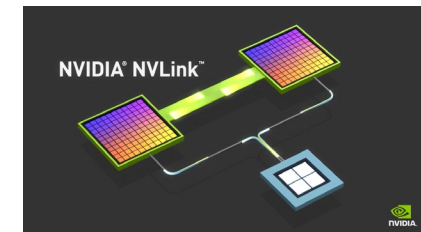
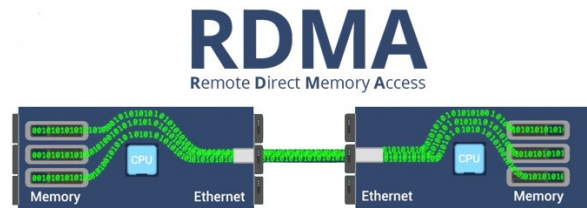


Custom Parallelization

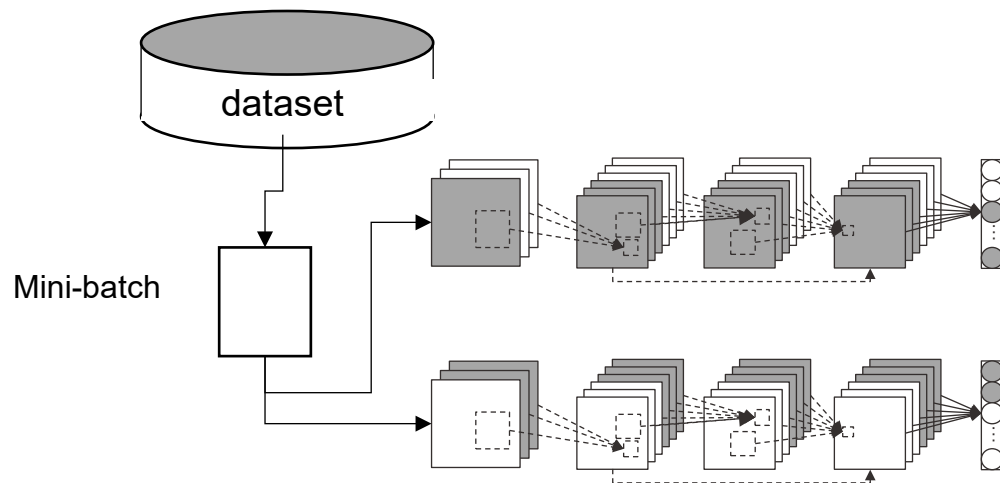
- Parallelization Software Libraries
 - OpenMP: Shared-Memory Programming Model (SMP)
 - MPI: Distributed-Memory Programming Model (DMP)
 - RDMA-based technologies (iWARP, RoCE, etc)
 - NVLink: DMP
 - Etc.

OpenMP

MPI

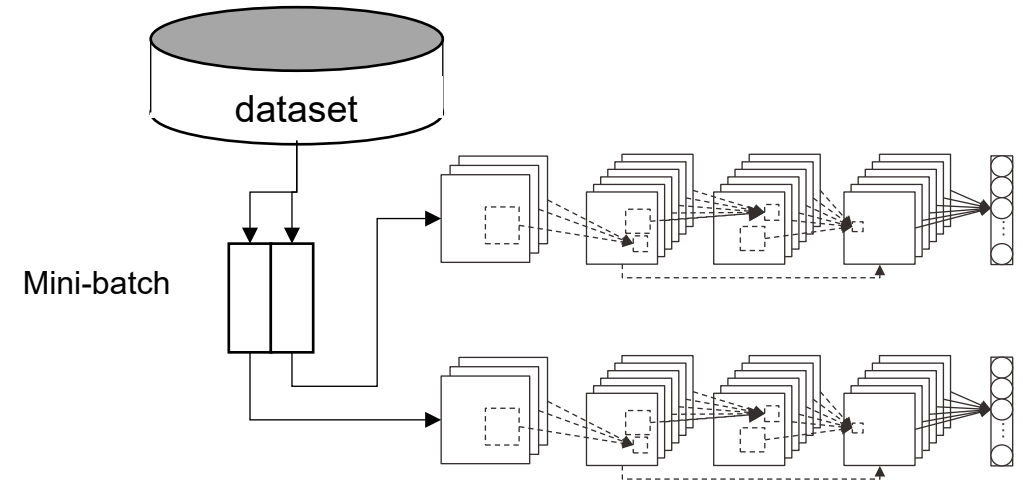


Model Parallelism vs. Data Parallelism



The model is distributed to different computing resources.

서로 다른 프로세스의 데이터에 대한 의존성 발생,
프로세스간 통신 빈번



The data is distributed to different computing resources.

서로 다른 프로세스의 데이터에 대한 의존성 없음,
프로세스간 통신 없음

Comparison

- Model Parallelism
 - The model parameters are distributed to all workers.
 - The data are replicated by all the workers.
 - Usually, it is tricky to implement the model parallelism especially when the model is complex.
- Data Parallelism
 - The data are distributed to all the workers.
 - The model parameters are replicated.
 - It is easy to implement because the whole model is simply replicated.

Data Parallelism is Dominant

- Due to its simplicity in implementation and the relatively low synchronization cost, data parallelism is used in general.
- E.g., Parallel K-Means
 - The data points are distributed to and locally processed within multiple processes.

Algorithm: Parallel K-Means

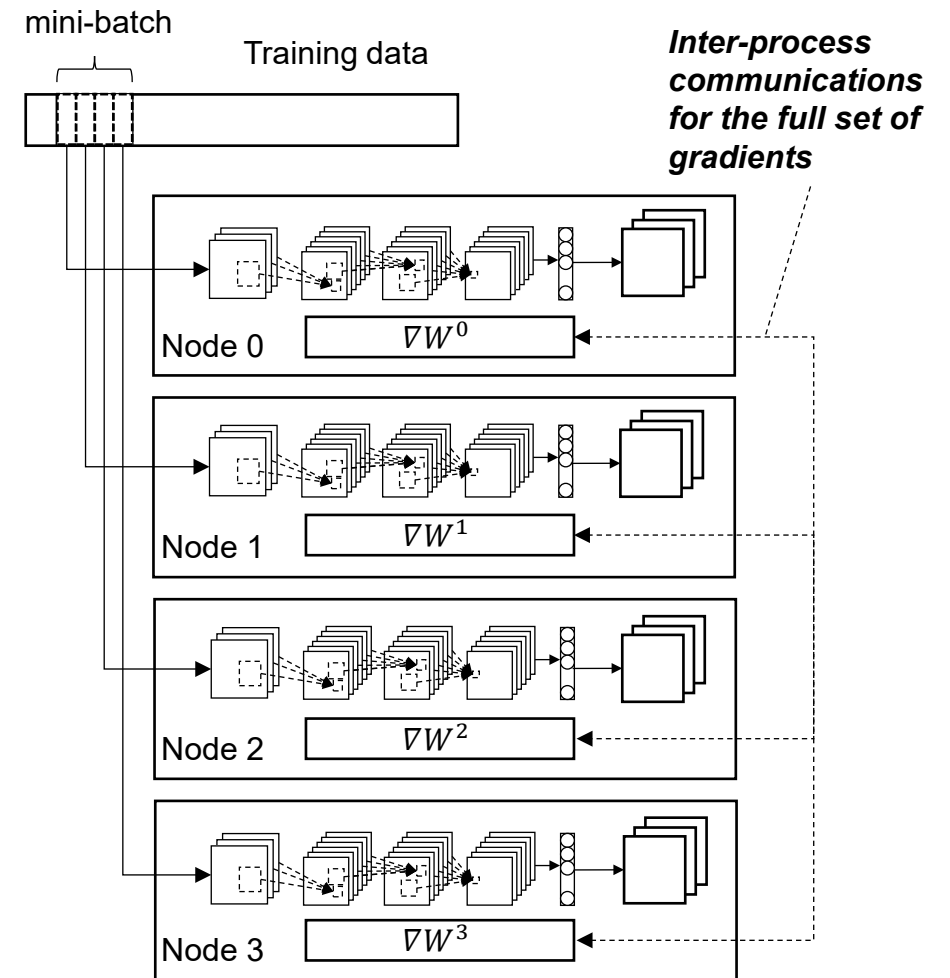
1. Initialize the centroids.
2. while (termination condition):
3. *Equally distribute the data to all the processes.*
4. Update the membership of the assigned data points.
5. Recalculate the centroids.
6. *Average the local centroids across all the processes.*

Large-Scale Deep Learning

- Synchronous SGD with data parallelism is the most popular scaling strategy for Deep Learning.
- The communication cost depends only on the model size.
- Only one synchronization point per training iteration.

$$w_{t+1} = w_t - \eta \underbrace{\frac{1}{p} \sum_{i=1}^p \nabla F_i(w_t)}_{\text{This summation is implemented with communications.}}$$

This summation is implemented with communications.

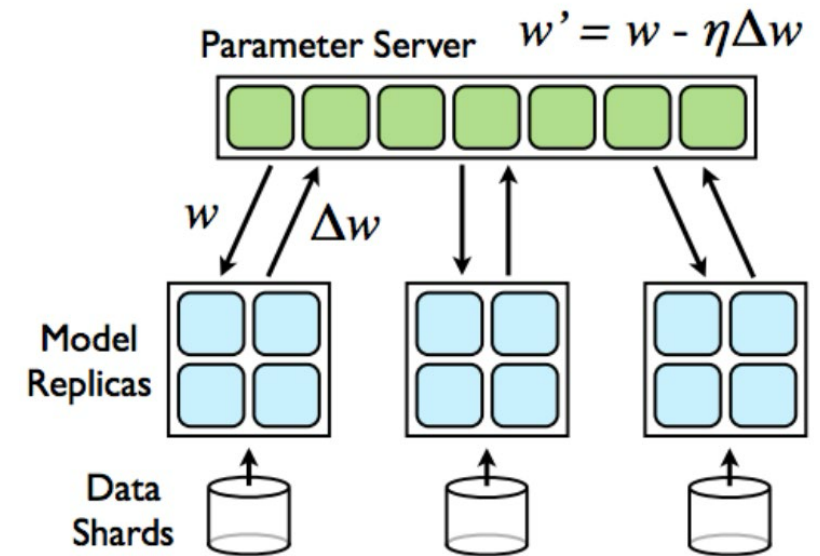


Open Problems

- The expensive *communication* cost issue
 - How could we achieve linear speedup?
- The expensive *computational* cost issue
 - How could we exploit the given hardware resources?
- *Trade-off* between training (statistical) efficiency and system efficiency
 - Some hyper-parameter settings allow a good system efficiency while degrading the classification performance.
 - E.g. How to find a good batch size in mini-batch SGD

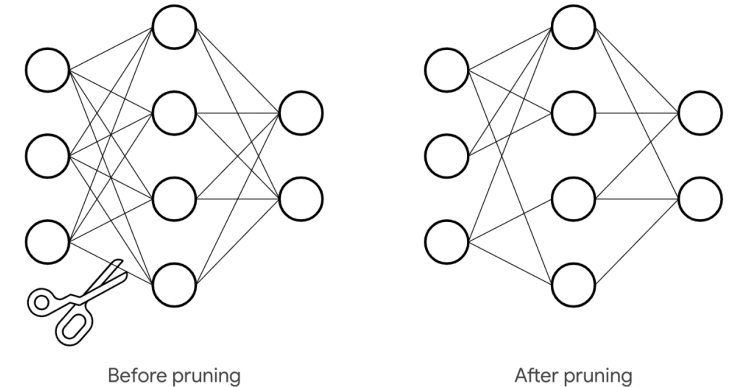
How to Mitigate Communication Cost

- Communication-Efficient Deep Learning
 - **Asynchronous** SGD
 - E.g., Do not wait until all the gradients are aggregated, and just update the model using the gradients received within a certain time period.
 - Gradient **Sparsification** / **Compression** / **Approximation**
 - Reduce the gradient quality while reducing the communication cost.
 - E.g., SVD-based gradient approximation or lossy compression algorithms
 - Computation-Communication **Overlapping**
 - Neural networks have layer-wise data dependency.
 - The communication at one layer can be overlapped with the computation at the next layer.



How to Reduce Computational Cost

- Model **Pruning** / **Compression**
 - E.g., Drop a few less important weight parameters to reduce both computational and communication costs.
 - Potentially harm the learning capability.
- **Low-Precision** Floating-Point Numbers
 - Instead of using full precision (32-bit floats), use half-precision or even bits to represent the data.
 - As long as the data is represented in a robust way, it works!
- Model Architecture **Search**
 - Find a model architecture that provides good learning capability while its size is small.



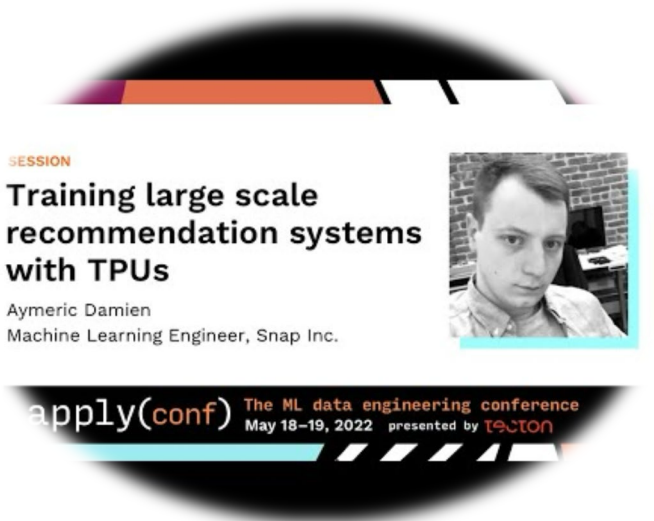
- Why Large-Scale?
- Scaling: How and What?
- Scaling ML Solutions
- ***Real-World Large-Scale ML***

Real-World Large-Scale ML Solutions

- The available data volume is getting larger every day.
- Many applications employ ML solutions to tackle the important real-world large-scale problems.
 - Recommendation System
 - SNS Analysis
 - Language Modeling
 - Scientific Applications (Physics, Cosmology, Biology, etc.)

Large-Scale Recommendation Systems

- Many IT companies such as Google, Facebook, Amazon develop their own product-level recommendation systems.
- Thanks to their abundant data, they usually can build a huge recommendation system.
 - E.g., Google uses their TPU-based supercomputers.



Large Language Models (1/2)

- ChatGPT employed GPT-5 which is not open to public.
- GPT-4.5 has several billions of parameters to train.

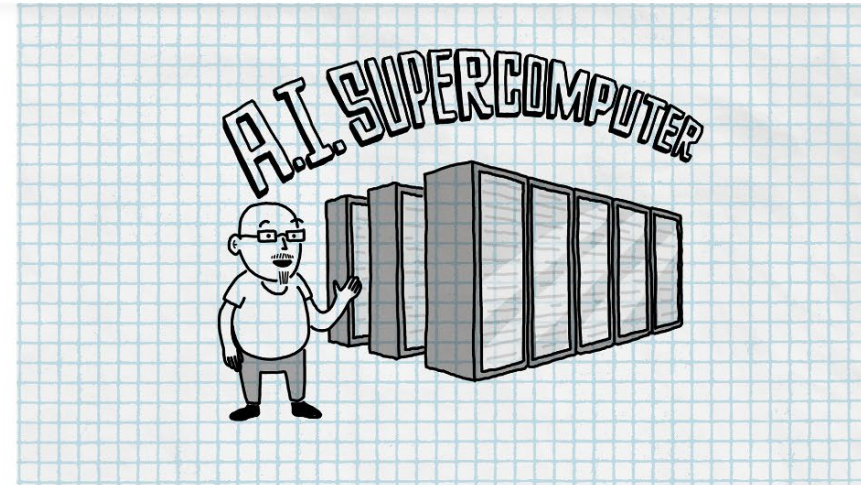
Microsoft | Source



May 19, 2020

Microsoft announces new supercomputer, lays out vision for future AI work

By [Jennifer Langston](#)



Microsoft has built one of the top five publicly disclosed supercomputers in the world, making new infrastructure available in Azure to train extremely large artificial intelligence models, the company is announcing at its Build developers conference.

Built in collaboration with and exclusively for [OpenAI](#), the supercomputer hosted in Azure was

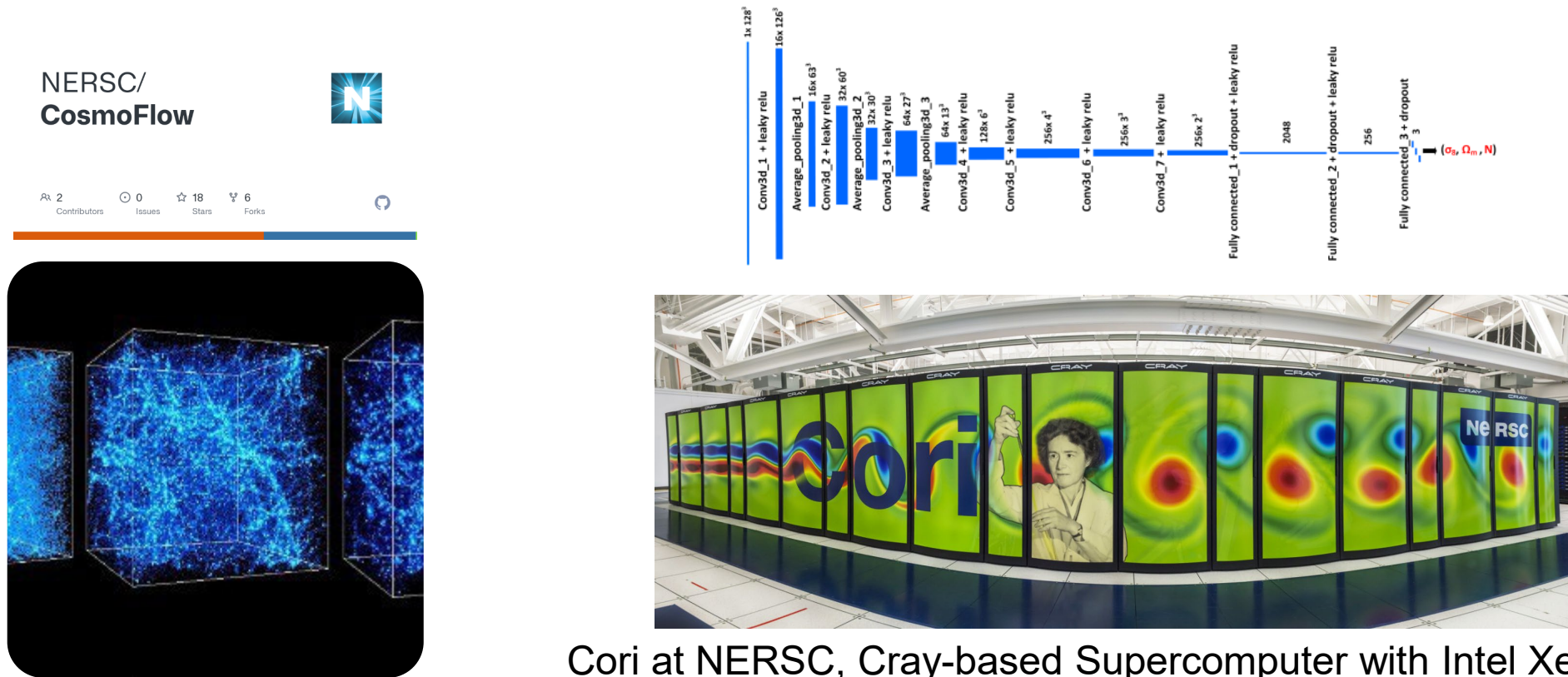
Large Language Models (2/2)

- It is wise to 'use' them rather than get 'caught up' in them.
- Remember, LLMs are not perfect! It's guessing or generating 'good' estimations!



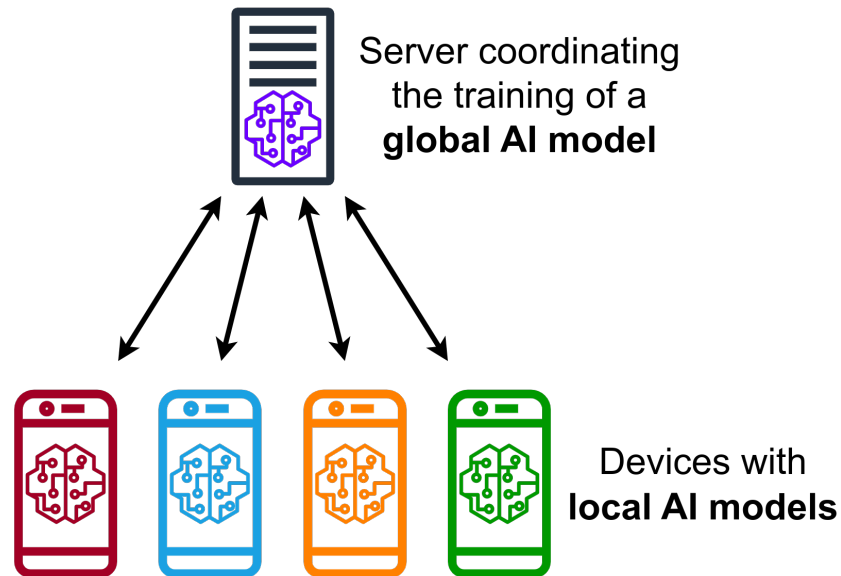
ML for Large-Scale Scientific Applications

- CosmoFlow
 - A large-scale deep learning-based Cosmology simulation data analysis.



Federated Learning (1/2)

- A new paradigm of distributed learning.
- Google announced this new paradigm in AISTATS 2017.



Communication-Efficient Learning of Deep Networks from Decentralized Data

H. Brendan McMahan¹ Eider Moore² Daniel Ramage³ Seth Hampson⁴ Blaise Agüera y Arcas¹
¹Google, Inc., 651 N 34th St., Seattle, WA 98103 USA

Abstract

Modern mobile devices have access to a wealth of data suitable for learning models, which in turn can greatly improve the user experience on the device. For example, language models can improve speech recognition and text entry, and image models can automatically select good photos. However, this rich data is often privacy sensitive, large in quantity, or both, which may preclude logging to the data center and training there using conventional approaches. We advocate an alternative that leaves the training data distributed on the mobile devices, and learns a shared model by aggregating locally-computed updates. We term this decentralized approach *Federated Learning*.

We present a practical method for the federated learning of deep networks based on iterative model averaging, and conduct an extensive empirical evaluation, considering five different model architectures and four datasets. These experiments demonstrate the approach is robust to the unbalanced and non-IID data distributions that are a defining characteristic of this setting. Communication costs are the principal constraint, and we show a reduction in required communication rounds by 10–100× as compared to synchronized stochastic gradient descent.

1 Introduction

Increasingly, phones and tablets are the primary computing devices for many people [30, 2]. The powerful sensors on these devices (including cameras, microphones, and GPS), combined with the fact they are frequently carried, means they have access to an unprecedented amount of data, much of it private in nature. Models learned on such data hold the

promise of greatly improving usability by powering more intelligent applications, but the sensitive nature of the data means there are risks and responsibilities to storing it in a centralized location.

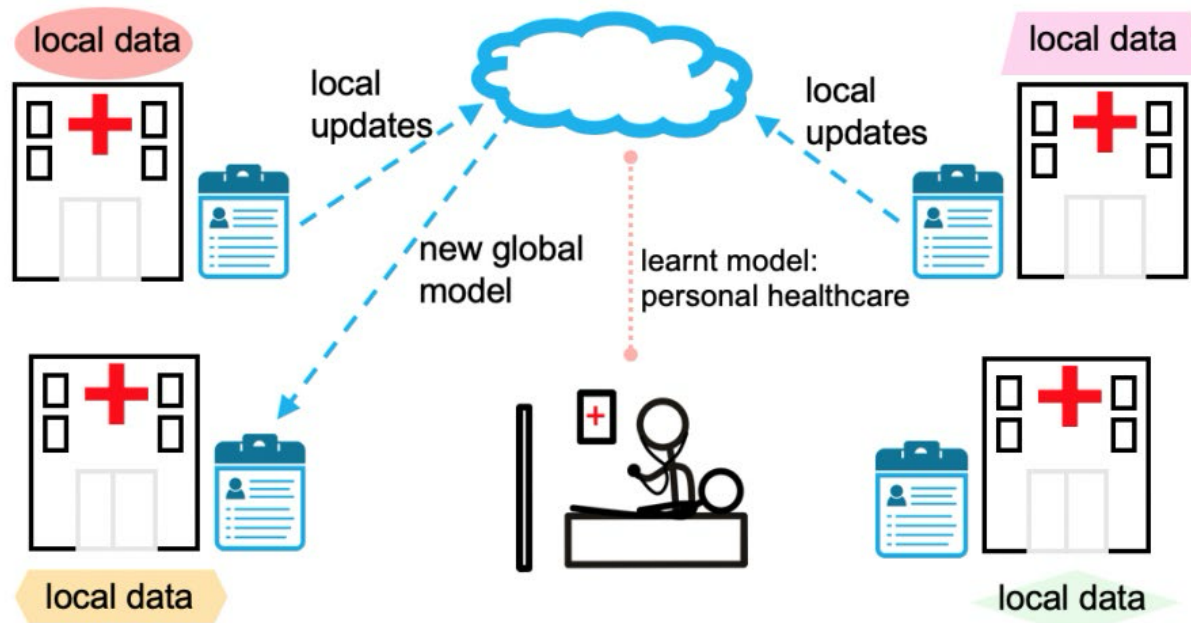
We investigate a learning technique that allows users to collectively reap the benefits of shared models trained from this rich data, without the need to centrally store it. We term our approach *Federated Learning*, since the learning task is solved by a loose federation of participating devices (which we refer to as *clients*) which are coordinated by a central *server*. Each client has a local training dataset which is never uploaded to the server. Instead, each client computes an update to the current global model maintained by the server, and only this update is communicated. This is a direct application of the principle of *focused collection or data minimization* proposed by the 2012 White House report on privacy of consumer data [39]. Since these updates are specific to improving the current model, there is no reason to store them once they have been applied.

A principal advantage of this approach is the decoupling of model training from the need for direct access to the raw training data. Clearly, some trust of the server coordinating the training is still required. However, for applications where the training objective can be specified on the basis of data available on each client, federated learning can significantly reduce privacy and security risks by limiting the attack surface to only the device, rather than the device and the cloud.

Our primary contributions are 1) the identification of the problem of training on decentralized data from mobile devices as an important research direction; 2) the selection of a straightforward and practical algorithm that can be applied to this setting; and 3) an extensive empirical evaluation of the proposed approach. More concretely, we introduce the *Federated Averaging* algorithm, which combines local stochastic gradient descent (SGD) on each client with a server that performs model averaging. We perform extensive experiments on this algorithm, demonstrating it is robust to unbalanced and non-IID data distributions, and can reduce the rounds of communication needed to train a deep network on decentralized data by orders of magnitude.

Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS) 2017, Fort Lauderdale, Florida, USA. JMLR: W&CP volume 54. Copyright 2017 by the author(s).

Federated Learning (2/2)

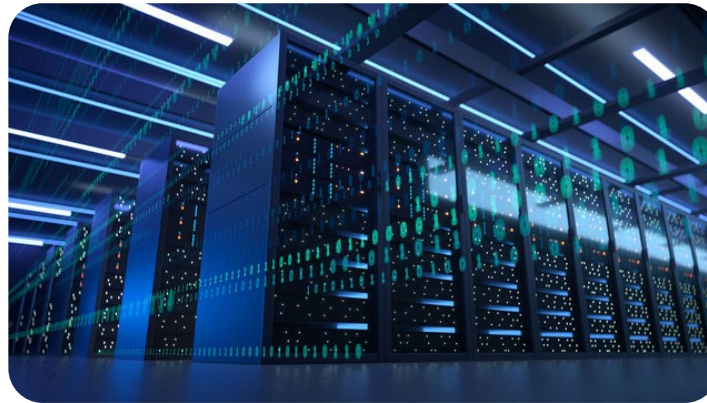


$$w_{t+1} = w_t - \frac{\eta}{C} \sum_{i=1}^C \sum_{j=1}^{\tau} \nabla F_i(w_{t,j})$$

Each client locally updates the model τ times, and then the model is aggregated at the server.

Final Notes

- In this BigData era, the important thing is how to make a good use of a large amount of data rapidly.
- AlphaGo, YouTube Recommender Systems, and ChatGPT have shown the potential of large-scale ML.
- There are many interesting and important problems that are still open!



Any Questions?

Thank you for your attention.

Sunwoo Lee

<sunwool@inha.ac.kr>